Pathology Voice Reporting — Streaming Architecture

AssemblyAI → FastAPI/WebSocket → LangGraph → LLM → Pydantic → Live UI

1. Core Behavior

The clinician does not have to dictate the report in a fixed order. AssemblyAI produces transcript chunks after pauses in speech. Each chunk is processed while the system keeps the accumulated report state.

Doctor speaks	→	AssemblyAI	→	Transcript chunk
Transcript chunk	→	FastAPI / LangGraph	→	LLM understands it
LLM	→	Target section(s)	→	Structured update
Update	→	Pydantic	→	Validated report state
State update	→	WebSocket	→	Live UI

2. Key Architecture Principle

Report sections are state, not sequential LangGraph nodes. Do not force Patient → Specimen → Gross → Microscopic → Diagnosis. The doctor can start with any section, skip sections, return later, or discuss multiple sections in one chunk.

LangGraph should contain processing operations:

```
Receive chunk → Understand chunk → Determine target fields → Create update → Validate → Merge into state → Send UI update → Wait for next chunk
```

3. Persistent Report State

The current report is the graph's working memory. Every new chunk is interpreted against this state.

```
report_state = { "patient": {}, "specimen": {}, "gross_description": "", "microscopic_description": "", "diagnosis": "", "comments": "", "active_section": None, "recent_context": [] }
```

4. How Each Chunk Is Processed

1	Receive	Get the new AssemblyAI transcript chunk.
2	Understand	LLM identifies the clinical information in the chunk.
3	Route	LLM determines which report field(s) should change.
4	Update	LLM returns only structured changes, not the whole report.
5	Validate	Pydantic validates the proposed changes.
6	Merge	Backend merges validated data into the existing state.
7	Stream	WebSocket sends the update to the UI.

5. Example: Doctor Starts With Microscopic Findings

Chunk 1: "Microscopically, there is chronic inactive gastritis."

```
{ "updates": [{ "target_section": "microscopic_description", "operation": "append", "content":
"Chronic inactive gastritis." }] }
```

The microscopic field is updated even though specimen and gross description are still empty. No earlier section needs to execute first.

6. Second Chunk Is About the Same Section

Chunk 2: "No Helicobacter organisms are identified."

```
{ "updates": [{ "target_section": "microscopic_description", "operation": "append", "content": "No
Helicobacter organisms are identified." }] }
```

The backend appends the second chunk to the same report field. The LLM uses the existing microscopic content, active section, recent context, and the new chunk to understand the relationship.

Result:

```
Microscopic Description: "Chronic inactive gastritis. No Helicobacter organisms are identified."
```

7. Doctor Can Jump to Another Section

Chunk 3: "The specimen is a gastric biopsy from the antrum."

```
{ "updates": [{ "target_section": "specimen", "operation": "update", "content": { "type": "gastric
biopsy", "site": "antrum" } }] }
```

The specimen state is updated. The graph did not need to execute a specimen node earlier; it simply updated the specimen portion of the report state.

8. One Chunk Can Update Multiple Sections

If the clinician says: "The specimen is a gastric biopsy from the antrum. Microscopically there is chronic inactive gastritis."

```
{ "updates": [ {"target_section": "specimen", "operation": "update", "content": {"type": "gastric
biopsy", "site": "antrum"}}, {"target_section": "microscopic_description", "operation": "append",
"content": "Chronic inactive gastritis."} ] }
```

Therefore, the LLM should be able to return multiple updates from one transcript chunk.

9. How Context Is Maintained

The LLM should not receive only the newest chunk. It should receive the accumulated structured report plus limited recent context.

Context	Purpose
Current report	Long-term working memory of extracted information.
Active section	Hint about what the clinician is currently discussing.
Recent context	Short-term context for interpreting the new chunk.
New chunk	The new information to classify and merge.

10. Active Section Is a Hint, Not a Rule

If active_section is microscopic_description and the doctor says, 'The final diagnosis is chronic gastritis,' the LLM can override the active section and route the chunk to diagnosis.

Use active_section as supporting context, never as an absolute routing rule.

11. Recommended LLM Responsibility

Ask the LLM one narrow question: analyze the new transcript chunk, determine what information it contains, determine where it belongs in the existing report, and return structured updates.

```
INPUT - Current report state - Active section - Recent context - New transcript chunk OUTPUT - Target section(s) - Operation: update / append - Structured content - Optional confidence - No complete report regeneration
```

12. LangGraph Flow

```
START
↓
Receive AssemblyAI Chunk
↓
Understand Chunk
↓
Determine Target Section(s)
↓
Create Structured Update
↓
Pydantic Validation
↓
Merge Into Report State
↓
Send Update Through WebSocket
↓
WAIT FOR NEXT CHUNK
↓
(repeat)
```

13. Final Comment Generation

Generate the AI comment as a separate final step after the clinician indicates that the report is complete. Pass the accumulated validated report to the comment-generation LLM.

```
Validated Report → Comment Generator LLM → AI-generated Comment → Pydantic Validation → UI → Clinician Review → Finalization
```

Healthcare safety: AI-generated comments should be clearly labeled as AI-generated and reviewed by the clinician. The system must not silently invent clinical facts or present AI-generated text as clinician-authored.

14. End-to-End Architecture

Doctor	Speaks continuously
AssemblyAI	Streaming STT; transcript chunks after pauses
FastAPI	Maintains reporting session and WebSocket
LangGraph	Orchestrates chunk processing and state updates
LLM	Determines target fields and creates structured updates
Pydantic	Validates updates before state changes
Report State	Stores the complete accumulated report
WebSocket	Streams meaningful updates to frontend
UI	Updates the appropriate fields immediately
Finish	Generates AI comment from final validated report
Clinician	Reviews and finalizes

Core principle: Every AssemblyAI chunk enters the same LangGraph workflow. The LLM uses current report state + recent context + the new chunk to determine where the information belongs. The report can be filled in any order, repeated chunks can update the same field, and the UI receives validated changes immediately.